

MiniRedis

A Redis-style in-memory database built from scratch in C++20
Problem Statement and Objectives

1. The problem

Almost every application that serves users at scale sits in front of a database it cannot afford to hit for every request. A relational database answers a primary-key lookup in roughly 1-10 milliseconds, most of which is disk seek, query planning and connection overhead rather than the work of finding the row. An in-memory data store answers the same lookup in tens of microseconds - two to three orders of magnitude faster - because the data is already in RAM and the path between socket and value is short.

That is why Redis, Memcached and their relatives are near-universal infrastructure. Session stores, rate limiters, leaderboards, job queues, feature flags and caches are all built on them.

The problem this project addresses is not "we need another cache". It is that these systems are used as black boxes. An engineer can operate Redis for years without being able to answer:

- What actually happens between SET k v arriving on a socket and the reply going back out?
- Redis is single-threaded. Why is that fast rather than slow?
- If the process is killed mid-write, exactly how much data is lost, and what determines that number?
- A replica reconnects after a three-second network blip. Does it re-transfer the entire dataset, and what decides?
- The server hits its memory limit. Which key is deleted, and how was it chosen without scanning everything?
- MULTI/EXEC is called a transaction. Is it? What happens if the third of five commands fails?

These are not trivia. Each is a decision with a real engineering trade-off behind it, and each determines how the system behaves on the day something goes wrong. Reading the documentation gives you the what. Building the system gives you the why.

1.1 The concrete starting point

This project began as a small distributed key-value store: about 1,600 lines of C++ implementing a hash map behind a TCP socket, a write-ahead log, a snapshot mechanism, leader/follower replication and a small SQL parser. It worked, and it demonstrated the basic shape of the problem. It also had specific, instructive defects. A code review found six:

#	Defect	Consequence
1	setsockopt passed a 1-byte char for the 4-byte SO_REUSEADDR option	Three bytes of the option value came from uninitialised stack memory
2	The socket reader issued one recv() syscall per byte	Two orders of magnitude more syscalls than necessary per command
3	The write-ahead log was appended after the lock was released	Two concurrent writers to one key could log in the opposite order to the one they applied, so recovery restored the losing value
4	INSERT/UPDATE checked existence and wrote in two separate locked sections	Time-of-check-to-time-of-use race: two clients could both believe they created the same key

#	Defect	Consequence
5	The log's escaping encoded an empty value as an empty token	A record with an empty value read back with every later field shifted left
6	Accepted connections were pushed into a vector drained only after an infinite loop	Every connection ever accepted leaked a thread handle for the process lifetime

Defects 3, 4 and 6 are all consequences of one architectural choice: thread-per-connection with shared mutable state. That is the single most important thing the original codebase has to teach, and it motivates the rewrite.

2. Objectives

2.1 Primary objective

Build a production-shaped, Redis-compatible in-memory database from first principles - no database libraries, no networking frameworks, no serialisation libraries - such that every architectural decision in the system is one the author made deliberately and can defend.

Success criterion: the stock redis-cli connects to MiniRedis and works without modification. That is a falsifiable, externally-verifiable claim about protocol correctness which cannot be faked with a convincing-looking demo.

2.2 Specific objectives

	Objective	How it is measured
O1	Correct the six defects, and eliminate their root cause	Each fix lands as its own commit explaining the failure mode; the resulting server has no locks in the command path
O2	Implement a real wire protocol (RESP2)	redis-cli interoperates; a test feeds every prefix of a command and asserts the parser consumes zero bytes each time
O3	Implement Redis's data model	≥ 140 commands; WRONGTYPE on type mismatch; an emptied container is indistinguishable from a missing key
O4	Key expiry that is correct and bounded	Keys with elapsed TTLs are reclaimed without being accessed; cycle cost is bounded by sample size, not keyspace size
O5	Memory limits and eviction	Under a 2 MB limit with allkeys-lru, writing 4 MB succeeds and stays within the limit; noeviction refuses with OOM
O6	Both persistence strategies, and the trade-off	A kill -9 mid-workload loses no acknowledged write under appendfsync always; a corrupted snapshot is rejected; a truncated log tail is recovered from
O7	Replication that survives a reconnect cheaply	A replica reconnecting within the backlog window receives only the bytes it missed; INFO replication reports the offsets
O8	Horizontal sharding	CLUSTER KEYSLOT foo returns 12182, matching real Redis exactly, so cluster-aware clients route correctly
O9	Transactions and pub/sub, with precise guarantees	A WATCHed key modified before EXEC aborts the transaction; the documentation states what the transaction does not guarantee
O10	Measure it, rather than asserting it is fast	p50/p95/p99 latency and throughput across pipeline depths and concurrency levels, charted and interpreted
O11	Make the codebase legible	Every non-obvious decision carries a comment naming the rejected alternative; zero warnings under -Wall -Wextra; builds on Windows and Linux

2.3 Explicit non-objectives

Stating what is not attempted is part of an honest problem statement:

- Not a Redis replacement. Real Redis has fifteen years of optimisation, multiple memory encodings per type, Lua scripting, streams, HyperLogLog and modules.
- Not automatic failover. Coordinated failover requires consensus. Implementing it badly produces split brain, so it is deliberately omitted and the omission is documented at the point where a user would look for it.

- Not forked background saves. Real Redis BGSAVE forks and relies on copy-on-write; `fork()` does not exist on Windows. MiniRedis saves synchronously and says so in the reply rather than pretending.
- Not RESP3. The server advertises RESP2 and rejects a RESP3 handshake explicitly, rather than accepting it and then failing to send push frames.

3. Scope

Layer	Delivered
Network	Cross-platform non-blocking sockets (Winsock2 + POSIX), poll()/WSAPoll multiplexing
Protocol	RESP2 codec, incremental request parser, inline command support
Execution	Single-threaded event loop, command dispatch table with metadata-driven routing
Data model	Strings, lists, hashes, sets, sorted sets - roughly 150 commands
Lifetime	TTL expiry, lazy plus an active sampling cycle
Memory	maxmemory with 8 eviction policies, sampled approximated LRU/LFU
Durability	Append-only log with 3 fsync policies and self-rewriting; CRC64-checked binary snapshots
Distribution	Leader/follower replication with offsets, ring backlog, partial resync
Sharding	16384 hash slots, CRC16, MOVED redirection, hash tags
Concurrency control	MULTI/EXEC/DISCARD/WATCH optimistic locking
Messaging	Pub/Sub with channel and glob-pattern subscriptions
Observability	INFO, SLOWLOG, MONITOR, MEMORY DOCTOR, CLIENT LIST
Query layer	A small SQL surface over the string keyspace
Tooling	Server, interactive CLI, benchmark harness
Verification	C++ unit suite plus a process-level integration suite

Out of scope: Lua scripting, Redis Streams, HyperLogLog, geospatial indexes, bitfields, modules, TLS, ACL users, Sentinel, automatic cluster resharding, RESP3.

4. Approach

The work proceeds in three phases, and the commit history follows them exactly, so the repository reads as a narrative rather than a drop.

- Phase 1 - Repair. Fix each of the six defects as its own commit, with a message explaining the failure mode. This establishes what was wrong before anything is replaced, and makes the case for Phase 2.
- Phase 2 - Rebuild. Replace the architecture, one subsystem per commit, in dependency order: platform abstraction, protocol, data model, keyspace, event loop, persistence, replication, sharding, commands.
- Phase 3 - Verify and document. Unit tests for components, integration tests for behaviour that only appears across process boundaries, a benchmark harness for the performance claims, and written documentation for the reasoning.

4.1 The central design decision

The most consequential choice is replacing thread-per-connection with a single-threaded event loop, and it is worth stating the reasoning explicitly because it is counter-intuitive.

The intuition is that more threads means more throughput. For a CPU-bound workload that is true. For an in-memory database it is not, because the work per command is measured in hundreds of nanoseconds - a hash lookup and a memory copy. Against that, a mutex acquisition under contention costs comparable time, and a context switch costs an order of magnitude more. The synchronisation overhead is not amortised by the work; it

dominates it.

Removing threads buys three things: atomicity for free, since every command is atomic because nothing else runs concurrently - defects 3, 4 and 6 become unrepresentable; no synchronisation cost at all; and a simple concurrency story, because concurrency comes from multiplexing thousands of sockets with one `poll()` call, which is where the actual waiting happens.

The cost is real and is documented rather than hidden: one slow command blocks every client. That is precisely why KEYS is discouraged in favour of SCAN, why the slow log exists, and why DEBUG SLEEP is implemented - so the cost can be demonstrated rather than described.

5. Expected outcomes

- A working database that real Redis clients can talk to.
- A commit history that documents six real defects and their root cause.
- Measured performance data with latency distributions, not marketing numbers.
- A test suite covering both component correctness and cross-process behaviour.
- Written explanations of every significant trade-off, including the ones where MiniRedis chose differently from Redis, and why.

The deliverable is not only the binary. It is the ability to answer every question in section 1 from having built the answer.